Node.js — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Installation

02 npm and package.json

03 Modules: CommonJS and ES Modules

04 Core Modules

05 File System

06 Environment Variables and Process

Intermediate (6 topics)

07 Events and EventEmitter

08 Asynchronous Patterns

09 HTTP Server Without Frameworks

10 Streams and Buffers

11 Working with Databases

12 Error Handling and Logging

Advanced (6 topics)

13 The Event Loop and Performance

14 Worker Threads and Cluster

15 Authentication: JWT and Sessions

16 Security Best Practices

17 Testing

18 WebSockets and Real-time

Master (4 topics)

19	Architecture and Project Structure
20	Docker, Deployment and CI/CD
21	Monitoring and Scaling
22	Real Project: REST API with Auth and DB

Beginner Level

01. Introduction and Installation

DEFINITION

Node.js runs JavaScript outside the browser using the V8 engine, with a non-blocking I/O model. Install the LTS version and run files with `node`.

SYNTAX

```
node app.js
```

EXAMPLE

```
node -v
node app.js
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A power-house that runs JavaScript outside the browser, like a generator that lets you run a shop machine when there is no main power.

தமிழ்

மின்தடை நேரத்தில் கடை இயந்திரத்தை ஓட்டும் ஜெனரேட்டர் போல Node.js browser-க்கு வெளியே JavaScript-ஐ இயக்குகிறது.

PROS

- Same language on frontend and backend
- Fast I/O
- Huge npm ecosystem

CONS

- Not ideal for heavy CPU tasks
- Callback/async complexity
- npm dependency sprawl

ADVANCED / NEW FEATURES

- Built-in `fetch`, test runner, `--watch`, `--env-file`
- Native TypeScript type stripping
- `require(esm)` support

02. npm and package.json

DEFINITION

npm installs packages. `package.json` lists dependencies and scripts; `package-lock.json` pins versions.

SYNTAX

```
npm install pkg
```

EXAMPLE

```
npm init -y
npm i express
npm i -D nodemon
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A grocery supplier: you list what you need (package.json) and the supplier (npm) delivers exact items.

தமிழ்

மளிகை சப்ளையர்: தேவையான பொருட்களை பட்டியலிடுகிறீர்கள் (package.json), அவர் (npm) சரியானதை அனுப்புகிறார்.

PROS

- Millions of packages
- Scripts automate tasks
- Lockfile reproducibility

CONS

- Supply-chain risks
- Large `node_modules`

ADVANCED / NEW FEATURES

- npm 11
- `npm audit signatures`
- pnpm/yarn/bun alternatives

03. Modules: CommonJS and ES Modules

DEFINITION

Use `import/export` (set `"type": "module"`) or `require/module.exports`.

SYNTAX

```
import x from 'x';
const y = require('y');
```

EXAMPLE

```
import { readFile } from 'node:fs/promises';
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Different departments of a company exchange files; modules let files share only what they export.

தமிழ்

ஒரு நிறுவனத் துறைகள் கோப்புகளைப் பகிர்வது போல modules ஒவ்வொரு கோப்பும் export செய்ததை மட்டும் பகிர்கின்றன.

PROS

- Encapsulation
- Reuse

CONS

- CJS/ESM interop confusion

ADVANCED / NEW FEATURES

- `require(esm)`
- Stable ESM with `node:` imports

04. Core Modules

DEFINITION

`fs`, `path`, `os`, `url`, `crypto`, `http`, `events`, `util`, `stream`.

SYNTAX

```
import fs from 'node:fs';
```

EXAMPLE

```
import path from 'node:path';
console.log(path.join('a', 'b', 'c.txt'));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A toolbox that comes with the house: fs for files, path for addresses, http for the gate. Core modules ship with Node.

தமிழ்

வீட்டுடன் வரும் கருவிப்பெட்டி போல fs (கோப்பு), path (முகவரி), http (வாயில்) Node-உடன் வருகின்றன.

PROS

- No install needed
- Stable APIs

CONS

- Some APIs are low level

ADVANCED / NEW FEATURES

- `node:test`, `node:sqlite`, `fetch`, `WebSocket` built in

05. File System

DEFINITION

Read and write files with `fs/promises`; avoid blocking sync methods in servers.

SYNTAX

```
await fs.readFile(path, 'utf8')
```

EXAMPLE

```
import fs from 'node:fs/promises';
await fs.writeFile('a.txt', 'Hello');
console.log(await fs.readFile('a.txt', 'utf8'));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Reading and writing entries in a shop ledger. Async file APIs let the shop keep serving customers meanwhile.

தமிழ்

கடை கணக்குப் புத்தகத்தில் பதிவுகளை எழுதுவது/படிப்பது; async API கடை சேவையை நிறுத்தாமல் இதைச் செய்கிறது.

PROS

- Non-blocking
- Promise API

- Streams for big files

CONS

- Sync methods block the server
- Path/permission errors

ADVANCED / NEW FEATURES

- `fs.glob`
- Permission model

06. Environment Variables and Process

DEFINITION

Read config from `process.env`; use `.env` files with `--env-file` or dotenv; handle `process.argv`.

SYNTAX

```
process.env.NAME
```

EXAMPLE

```
node --env-file=.env app.js
console.log(process.env.PORT ?? 3000);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Keeping the shop key under a mat is risky; secrets belong in environment variables kept outside the code.

தமிழ்

கடைச் சாவியை பாய் கீழே வைப்பது ஆபத்து; ரகசியங்கள் குறியீட்டுக்கு வெளியே environment variables-இல் இருக்க வேண்டும்.

PROS

- Config separated from code
- Per-environment settings

CONS

- Easy to leak in logs

ADVANCED / NEW FEATURES

- `--env-file` built in (no dotenv needed)
- `process.loadEnvFile`

Intermediate Level

07. Events and EventEmitter

DEFINITION

Many Node objects emit events; create your own with `EventEmitter`.

SYNTAX

```
emitter.on('x', fn); emitter.emit('x');
```

EXAMPLE

```
import { EventEmitter } from 'node:events';
const e = new EventEmitter();
e.on('order', id => console.log('order', id));
e.emit('order', 7);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A temple bell: when it rings (event), all listening devotees react; EventEmitter notifies subscribers.

தமிழ்

கோயில் மணி அடித்தால் (event) கேட்கும் அனைவரும் பதிலளிப்பது போல EventEmitter சந்தாதாரர்களுக்கு அறிவிக்கிறது.

PROS

- Decoupled code
- Core Node pattern

CONS

- Memory leaks if listeners pile up
- Hard to trace flow

ADVANCED / NEW FEATURES

- `EventTarget` compatibility
- `events.once` promise helper

08. Asynchronous Patterns

DEFINITION

Callbacks, promises and `async/await`; `util.promisify`; never block the event loop.

EXAMPLE

```
const data = await fs.readFile('x.json', 'utf8');
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Like a token system: you take a token, do other work, and get called when ready; async patterns avoid blocking the line.

தமிழ்

டோக்கன் முறை போல: டோக்கன் பெற்று வேறு வேலை பார்த்து தயாரானதும் அழைக்கப்படுவீர்கள்; async முறைகள் வரிசையைத் தடுக்காது.

PROS

- High concurrency
- Simple with async/await

CONS

- Callback hell if misused

ADVANCED / NEW FEATURES

- Top-level await
- `AbortSignal` everywhere

09. HTTP Server Without Frameworks

DEFINITION

`http.createServer` handles requests and responses; frameworks like Express build on it.

SYNTAX

```
http.createServer((req, res) => { }).listen(3000)
```

EXAMPLE

```
import http from 'node:http';
http.createServer((req, res) => res.end('Hello')).listen(3000);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

The front desk of a hotel that greets each guest and sends a reply; a bare HTTP server is the foundation of every Node web framework.

தமிழ்

ஹோட்டல் வரவேற்பு மேசை ஒவ்வொரு விருந்தினரையும் வரவேற்று பதில் தருவது போல HTTP server ஒவ்வொரு Node framework-இன் அடிப்படை.

PROS

- Full control
- No dependencies

CONS

- Lots of manual routing/parsing

ADVANCED / NEW FEATURES

- HTTP/2 support
- Built-in `fetch` server-side

10. Streams and Buffers

DEFINITION

Streams process data in chunks (files, HTTP); `pipeline` connects them safely.

SYNTAX

```
a.pipe(b)
```

EXAMPLE

```
import { pipeline } from 'node:stream/promises';
import { createReadStream, createWriteStream } from 'node:fs';
await pipeline(createReadStream('in'), createWriteStream('out'));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Water flowing through a pipe from a tank to a bucket, instead of carrying the whole tank; streams move data in chunks.

தமிழ்

முழு தொட்டியையும் தூக்காமல் குழாய் வழியாக நீர் வாளிக்குச் செல்வது போல streams தரவை துண்டுகளாகக் கடத்துகின்றன.

PROS

- Low memory
- Backpressure handling

CONS

- Complex error handling
- Hard API surface

ADVANCED / NEW FEATURES

- Web Streams compatibility
- `stream/promises` `pipeline`

11. Working with Databases

DEFINITION

Use drivers/ORMs: `mysql2`, `pg`, `mongodb`/Mongoose, Prisma. Always use parameterised queries.

SYNTAX

```
await pool.query('SELECT ...', [id])
```

EXAMPLE

```
const [rows] = await pool.query('SELECT * FROM users WHERE id = ?', [id]);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop's back office ledger: the app asks questions to the database safely using prepared questions.

தமிழ்

கடையின் பின்புற பதிவேடு: செயலி prepared query மூலம் database-இடம் பாதுகாப்பாக கேள்வி கேட்கிறது.

PROS

- Many drivers/ORMs
- Connection pooling

CONS

- SQL injection if string concatenation

ADVANCED / NEW FEATURES

- `node:sqlite` built in
- Prisma/Drizzle modern ORMs

12. Error Handling and Logging

DEFINITION

Catch errors in async code, handle `unhandledRejection`, use structured logging (pino, winston).

SYNTAX

```
process.on('unhandledRejection', fn)
```

EXAMPLE

```
process.on('unhandledRejection', err => { console.error(err); process.exit(1); });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A first-aid box at the shop: errors will happen, so you plan for them and log properly.

தமிழ்

கடையில் முதலுதவிப் பெட்டி போல பிழைகள் வரும், அதனால் திட்டமிட்டு log செய்கிறோம்.

PROS

- Better debugging
- Safer production

CONS

- Excess logging costs money

ADVANCED / NEW FEATURES

- Structured logging with pino
- Diagnostics channel

Advanced Level

13. The Event Loop and Performance

DEFINITION

Phases: timers, poll, check, close; `process.nextTick` and microtasks run first. Avoid CPU work on the main thread.

SYNTAX

```
setImmediate(fn); process.nextTick(fn);
```

EXAMPLE

```
setImmediate(() => console.log('check'));
process.nextTick(() => console.log('tick'));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A cashier handles urgent customers first, then the normal queue; the event loop has priority phases.

தமிழ்

காசாளர் அவசர வாடிக்கையாளர்களை முதலில் கவனித்து பிறகு சாதாரண வரிசையைக் கவனிப்பது போல event loop-க்கு முன்னுரிமை நிலைகள் உண்டு.

PROS

- Efficient single thread
- Predictable ordering

CONS

- Starvation if nextTick is abused

ADVANCED / NEW FEATURES

- Better diagnostics for event loop lag

14. Worker Threads and Cluster

DEFINITION

`worker_threads` run CPU-heavy tasks in parallel; `cluster` or PM2 use all CPU cores.

SYNTAX

```
new Worker('./job.js')
```

EXAMPLE

```
import { Worker } from 'node:worker_threads';
new Worker('./heavy.js');
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Hiring extra helpers for a heavy task so the main counter is not blocked; worker threads and cluster use multiple cores.

தமிழ்

கனமான வேலைக்கு கூடுதல் உதவியாளர்களை நியமித்து முக்கிய கவுண்டர் தடைபடாமல் பார்ப்பது போல worker threads, cluster பல core-களைப் பயன்படுத்துகின்றன.

PROS

- Parallel CPU work
- Use all cores

CONS

- Extra memory
- Message passing complexity

ADVANCED / NEW FEATURES

- Piscina/worker pools
- Single executable apps

15. Authentication: JWT and Sessions

DEFINITION

Hash passwords with bcrypt/argon2, issue JWTs or session cookies, and verify on each request.

SYNTAX

```
jwt.sign(payload, secret, { expiresIn: '1h' })
```

EXAMPLE

```
const token = jwt.sign({ sub: user.id }, process.env.JWT_SECRET, { expiresIn: '1h' });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A temple entry wristband valid for a day: servers check the band instead of asking your name again.

தமிழ்

ஒரு நாள் செல்லுபடியாகும் கோயில் கை நாடா போல server நாடாவைப் பார்த்து மீண்டும் பெயர் கேட்காது.

PROS

- Stateless auth
- Works across services

CONS

- Token revocation is hard
- Leaked tokens are dangerous

ADVANCED / NEW FEATURES

- Passkeys/WebAuthn
- Argon2 hashing

16. Security Best Practices

DEFINITION

Validate input (Zod), rate limit, use Helmet headers, CORS configuration, keep dependencies patched, never commit secrets.

SYNTAX

```
app.use(helmet())
```

EXAMPLE

```
npm audit
npm i helmet cors express-rate-limit
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Locking doors and checking IDs even if the neighbourhood is safe; security headers and validation protect the app.

தமிழ்

பகுதி பாதுகாப்பாக இருந்தாலும் கதவைப் பூட்டி அடையாளம் சரிபார்ப்பது போல security headers, validation செயலியைப் பாதுகாக்கின்றன.

PROS

- Prevents common attacks
- Cheap to add

CONS

- Needs regular dependency updates

ADVANCED / NEW FEATURES

- `npm audit signatures`
- Permission model (`--permission`)

17. Testing

DEFINITION

Built-in `node:test` or Vitest/Jest; supertest for APIs; mock external services.

SYNTAX

```
import test from 'node:test';
```

EXAMPLE

```
import test from 'node:test';
import assert from 'node:assert';
test('adds', () => assert.equal(1 + 1, 2));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A factory quality check before dispatch; tests verify each function works.

தமிழ்

தொழிற்சாலை அனுப்பும் முன் தரச் சோதனை போல tests ஒவ்வொரு function-ம் சரியாக இயங்குகிறதா என்று சோதிக்கின்றன.

PROS

- Confidence
- Regression protection

CONS

- Time to write

ADVANCED / NEW FEATURES

- Built-in test runner with coverage
- Snapshot testing in `node:test`

18. WebSockets and Real-time

DEFINITION

Use `ws` or Socket.IO for chat, notifications and live dashboards.

SYNTAX

```
io.on('connection', socket => { })
```

EXAMPLE

```
io.on('connection', socket => socket.on('msg', m => io.emit('msg', m)));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A phone call instead of sending letters: both sides can speak any time; WebSockets give real-time chat and live updates.

தமிழ்

கடிதங்களுக்குப் பதிலாக தொலைபேசி அழைப்பு போல இருபுறமும் எப்போது வேண்டுமானாலும் பேசலாம்; WebSockets நேரடி chat, live updates தருகின்றன.

PROS

- Real-time updates
- Low latency

CONS

- Stateful connections are harder to scale

ADVANCED / NEW FEATURES

- Built-in WebSocket client
- Server-Sent Events alternative

Master Level

19. Architecture and Project Structure

DEFINITION

Layers: routes, controllers, services, repositories; dependency injection, configuration, and clean separation of concerns.

EXAMPLE

```
src/
  routes/ controllers/ services/ repositories/ config/
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A hospital with reception, doctors, pharmacy: each layer does one job. Layered architecture does the same for code.

தமிழ்

மருத்துவமனையில் வரவேற்பு, மருத்துவர், மருந்தகம்: ஒவ்வொரு அடுக்கும் ஒரு வேலை; layered architecture குறியீட்டுக்கும் அதையே செய்கிறது.

PROS

- Maintainable code
- Easy testing

CONS

- More files/boilerplate

ADVANCED / NEW FEATURES

- Frameworks like NestJS/Fastify popularise structure

20. Docker, Deployment and CI/CD

DEFINITION

Containerise Node apps with multi-stage Dockerfiles; run behind Nginx; use PM2/systemd; deploy via CI/CD.

SYNTAX

```
FROM node:22-alpine
```

EXAMPLE

```
FROM node:22-alpine
WORKDIR /app
```

```
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
CMD ["node", "src/server.js"]
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Shipping goods in standard containers so any port can handle them; Docker images run the same everywhere.

தமிழ்

பொருட்களை தரப்படுத்தப்பட்ட கொள்கலன்களில் அனுப்புவது போல Docker images எங்கும் ஒரே மாதிரி இயங்குகின்றன.

PROS

- Consistent environments
- Easy scaling

CONS

- Image size and security patching

ADVANCED / NEW FEATURES

- Node 22/24 LTS images
- Distroless images

21. Monitoring and Scaling

DEFINITION

Health checks, metrics (Prometheus), tracing (OpenTelemetry), caching with Redis, queues (BullMQ), horizontal scaling.

EXAMPLE

```
const queue = new Queue('emails'); await queue.add('welcome', { to });
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A watchtower checks traffic and alerts; monitoring, caching and queues keep a growing service healthy.

தமிழ்

கண்காணிப்புக் கோபுரம் போக்குவரத்தைக் கவனித்து எச்சரிப்பது போல monitoring, caching, queues வளரும் சேவையை ஆரோக்கியமாக வைக்கின்றன.

PROS

- Fewer outages
- Better performance

CONS

- Operational complexity

ADVANCED / NEW FEATURES

- OpenTelemetry standard
- Redis/BullMQ queues

22. Real Project: REST API with Auth and DB

DEFINITION

CRUD API with validation, JWT auth, PostgreSQL/MongoDB, tests, Docker and deployment.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A complete small business backend: users, orders, payments APIs in one project.

தமிழ்

பயனர்கள், ஆர்டர்கள், கட்டணம் API-கள் அடங்கிய முழுமையான சிறு வணிக backend.

PROS

- Real-world practice

CONS

- Takes time

ADVANCED / NEW FEATURES

- Use Node LTS, TypeScript, Fastify/Express 5

Interview Question Bank

Q1. What is Node.js?
A JavaScript runtime built on V8 with an event-driven, non-blocking I/O model.

Q2. Explain the event loop.
It processes callbacks in phases so a single thread can handle many concurrent I/O operations.

Q3. CommonJS vs ES Modules?
CommonJS uses `require` (synchronous); ESM uses `import` (static, async-capable).

Q4. What are streams?
Interfaces for processing data piece by piece, reducing memory usage.

Q5. `process.nextTick` vs `setImmediate`?
`nextTick` runs before the event loop continues; `setImmediate` runs in the check phase.

Q6. How do you scale Node.js?
Cluster/PM2 on multiple cores, load balancers, worker threads, caching and horizontal scaling.

Q7. How do you handle errors in async code?
`try/catch` with `await`, `.catch()` on promises and a global handler as a safety net.

Q8. What is middleware?
A function in the request pipeline with access to request, response and `next`.

Q9. How do you secure a Node app?
Validation, hashing, HTTPS, Helmet, CORS, rate limiting, dependency audits.

Q10. What is the difference between `dependencies` and `devDependencies`?
Runtime packages vs tools used only in development/build.

Capstone Project Ideas

1. CLI notes app
2. REST API for a library
3. Real-time chat
4. File upload service
5. URL shortener with Redis